Part 1: Implementing Mutexes

CSE113 Homework 2 - Report draft | Brian Lee | October 14, 2026

Design

The filter lock stores each thread's current level and one victim for each level. A thread moves through levels 1 through N-1. At each level it names itself the victim, then waits while another thread is at least at that level and it is still the victim. Changing its level to zero releases the lock. Each thread has a distinct ID, so only that thread writes its own level entry.

The bakery lock stores a choosing flag and a ticket for each thread. A thread sets its choosing flag, scans all tickets, chooses one larger than the maximum, and clears the flag. It waits for every other thread to finish choosing, then waits for any smaller (ticket, thread ID) pair. IDs break ties when two threads choose the same ticket. A zero ticket means the thread is not competing for the lock.

All shared algorithm entries are atomic and are accessed only through load() and store(), using the default sequentially consistent ordering. There are no atomic read-modify-write operations in either implementation. The number of threads and array pointers are ordinary variables because initialization finishes before workers start. Both submitted headers call std::this_thread::yield() in their waiting loops.

Function	Purpose, inputs, outputs, and assumptions
mutex()	No input. Initializes count to zero and pointers to null; the object is not ready to lock until init().
init(N)	Allocates and initializes arrays for N > 0 threads. Returns void. Called with no threads using the mutex.
lock(id)	Returns void with exclusive access. Requires a unique ID in [0, N-1], initialization, no recursion, and no bakery ticket overflow.
unlock(id)	Returns void after releasing the caller's lock by clearing its level/ticket. Requires a matching lock(id).
~mutex()	Deletes the arrays after all users finish. These objects are not copied, and IDs are not shared between live threads.

Experiment workflow and verification

Each configuration ran three times without ThreadSanitizer. Both the host and course container expose 10 CPUs, so Part 1 uses 10, 20, and 40 threads. The benchmark uses its original nominal one-second loop and 501-acquisition warmup. No throughput trials were excluded. Temporary header copies remove only yield() for the no-yield experiments; submitted headers retain it.

Native measurements use macOS on ARM64; temporary benchmark copies omit the Linux-only affinity function. Docker measurements use the existing reeselevine/cse113:latest ARM64 course image with 10 CPUs and the original wrappers, including their affinity calls. Compiler versions and raw output are saved in results/native.json and results/docker.json. Builds use the starter's language standards and no optimization flag.

Focused tests passed natively and in Docker. Clang ThreadSanitizer passed all six focused tests and all six supplied custom-implementation wrappers in Docker. The temporary sanitizer container used --security-opt seccomp=unconfined because the default syscall filter blocked sanitizer startup. The original make all, make test, and stack sequential builds also succeeded in Docker; all three supplied sequential checks reported success.

Part 1: Results

Throughput is the mean reported total number across three runs. CV is the population standard deviation of a run's per-thread counts divided by their mean, times 100; the table averages the three CV percentages. Lower CV means more even acquisition counts. N/A means all counts are zero, making CV undefined.

Course Docker container

Threads	Mutex	Spin behavior	Mean throughput	Mean CV (%)
10	C++	Baseline	8,024,975	3.73
10	Filter	No yield	546,500	0.89
10	Bakery	No yield	1,748,227	1.02
10	Filter	Yield	626,641	0.79
10	Bakery	Yield	1,889,544	0.21
20	C++	Baseline	5,888,350	16.40
20	Filter	No yield	71,742	15.27
20	Bakery	No yield	245	4.14
20	Filter	Yield	437,200	3.72
20	Bakery	Yield	696,883	0.02
40	C++	Baseline	5,962,545	23.44
40	Filter	No yield	1,214	307.58
40	Bakery	No yield	0	N/A
40	Filter	Yield	170,318	7.32
40	Bakery	Yield	165,444	0.19

Native macOS

Threads	Mutex	Spin behavior	Mean throughput	Mean CV (%)
10	C++	Baseline	8,133,303	0.87
10	Filter	No yield	960,811	1.75
10	Bakery	No yield	2,628,257	1.16
10	Filter	Yield	792,121	0.85
10	Bakery	Yield	915,447	0.04
20	C++	Baseline	8,447,260	4.60
20	Filter	No yield	78,653	9.64
20	Bakery	No yield	188	138.44
20	Filter	Yield	133,955	1.44
20	Bakery	Yield	113,339	0.02
40	C++	Baseline	8,140,249	8.87
40	Filter	No yield	41	81.96
40	Bakery	No yield	0	N/A
40	Filter	Yield	48,067	1.13
40	Bakery	Yield	43,705	0.03

The C++ mutex baseline is reused for both spin comparisons because its code does not change. The tables report benchmark counts, not a separately measured wall-clock rate: finishing a pending acquisition can extend execution beyond the nominal interval.

Part 1: Histogram Data and Analysis

Results: representative histograms

Per-thread acquisition counts from the first Docker trial at 10 threads:

Thread ID	C++	Filter spin	Bakery spin	Filter yield	Bakery yield
0	756,249	53,634	175,521	62,920	183,572
1	773,348	54,011	175,420	63,724	183,455
2	760,801	53,674	175,180	63,526	183,560
3	908,305	53,925	175,161	63,921	183,503
4	760,003	53,742	174,967	63,956	182,954
5	938,123	53,883	174,855	63,956	183,447
6	774,553	53,764	174,512	63,639	183,238
7	767,574	53,315	173,860	64,723	183,303
8	751,427	54,003	173,647	63,027	183,108
9	786,778	53,632	169,813	63,708	182,486

A high-spread case is Docker filter/no-yield with 40 threads, trial 1. Its full histogram (thread IDs 0-39) is: 3, 4, 3, 4, 3, 3, 4, 3, 3, 3, 4, 3, 4, 4, 4, 4, 3, 4, 4, 3, 3, 4, 3, 4, 4, 4, 3, 3, 3, 4, 4, 4, 5, 5, 3, 151, 1187, 3, 4, 5.

For a manual sensitivity check, removing the largest bin (thread 36, 1187 acquisitions) changes CV from 502.97% to 314.45%. This diagnostic does not change the reported throughput or remove any benchmark trial.

Analysis

The C++ mutex has much higher throughput than the custom locks in the Docker runs. The custom algorithms scan shared arrays, and the filter lock repeats that work at N-1 levels. At 10 threads, however, the custom locks generally distribute acquisitions more evenly. Increasing the thread count makes spinning especially costly: the scheduler can pause a thread that other threads need to advance. Yielding gives waiting threads an opportunity to let runnable competitors proceed, which greatly improves the oversubscribed custom-lock results. The native table also shows that the effect depends on the host scheduler; yield is not a guarantee of better throughput in every configuration.

The very large filter/no-yield CV at 40 threads is consistent with uneven scheduling and staggered worker startup. The histogram sensitivity check changes the numerical CV but does not erase the underlying variability; dropping a busy thread can also hide real unfairness. Other likely sources of variation are background activity, the host's different CPU core types, and the short measurement period. The bakery/no-yield Docker runs at 40 threads record zero measured acquisitions: the total acquisitions remain within the 501-event warmup, so zero does not prove deadlock and cannot be described as perfect fairness. I retained all trials, including these zeros and the large histogram bins, to keep the comparison transparent.

Part 2: A Fair Reader-Writer Lock

Design

The fair lock follows the default lock's structure and adds a waiting_writers counter. One internal std::mutex protects the active-reader count, writer flag, and waiting-writer count. A reader may enter only when no writer is active or waiting. A writer registers itself before waiting, then enters after existing readers leave and no writer is active. Registering a writer closes the gate to new readers, allowing the current readers to drain. When there are no writers, readers follow the default path with only one additional counter check; no condition variables or extra reader-side locks are used. This is writer preference, not a FIFO guarantee: continuous writers can delay readers, and writer ordering depends on scheduling.

Function	Purpose, inputs, outputs, and assumptions
rw_mutex()	No input. Initializes reader and waiting-writer counts to zero and writer to false.
lock_reader()	No input; returns void once shared access is acquired. Multiple readers can enter. Caller must not already own this lock.
unlock_reader()	No input; returns void after decrementing the active-reader count. Requires a matching reader acquisition.
lock()	No input; returns void with exclusive writer access. Registers one waiting writer, then removes that registration when admitted.
unlock()	No input; returns void after clearing the active-writer flag. Requires a matching writer acquisition.

Results

Each configuration ran three times. The table gives mean reader/writer counts and each count's minimum-to-maximum range. Values are the original wrapper's reported totals; counters are incremented before the lock calls and exclude the wrapper's 2,048-event warmup per role.

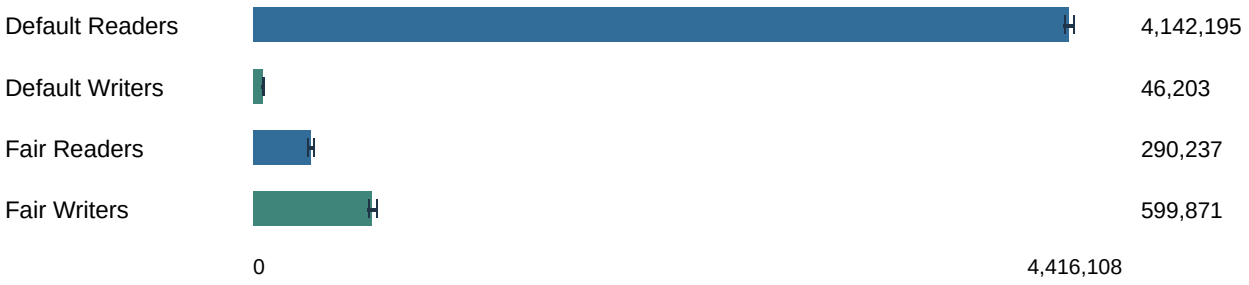
Environment	R/W	Mutex	Readers: mean (min - max)	Writers: mean (min - max)
Docker	6/2	Default	4,142,195 (4,118,393 - 4,166,140)	46,203 (44,399 - 48,631)
Docker	6/2	Fair	290,237 (276,321 - 302,800)	599,871 (582,521 - 624,861)
Docker	8/0	Default	5,678,618 (5,614,567 - 5,786,030)	0 (0 - 0)
Docker	8/0	Fair	5,676,893 (5,629,235 - 5,737,669)	0 (0 - 0)
Native	6/2	Default	3,612,924 (3,603,752 - 3,621,817)	89,938 (89,705 - 90,122)
Native	6/2	Fair	448,319 (440,674 - 454,086)	673,304 (672,062 - 675,761)
Native	8/0	Default	4,921,018 (4,901,030 - 4,933,633)	0 (0 - 0)
Native	8/0	Fair	4,981,522 (4,934,807 - 5,010,900)	0 (0 - 0)

Part 2: Graph and Analysis

Results: six readers and two writers

Grouped bars show mean reported counts; error bars show the minimum and maximum across three runs. Each panel has its own horizontal scale, and both locks use the same scale within a panel.

Course Docker



Native macOS

Analysis

Docker: writer counts increase 12.98 times, while the reader-only mean changes by -0.03%; Native: writer counts increase 7.49 times, while the reader-only mean changes by +1.23%. The much larger writer count and lower reader count in the mixed workload agree with the design: readers cannot repeatedly bypass waiting writers. In the no-writer workload, waiting_writers remains zero, so shared reader admission stays close to the default behavior. Short runs and the reported ranges limit how precisely a small reader-only difference can be interpreted. The lock improves access for the writer group but does not measure fairness between individual writers or prove a starvation bound. Native and Docker absolute rates differ because their mutex implementations and scheduling environments differ, and native affinity was unavailable.

Verification and reproduction

The focused reader-writer test first requires two readers to hold the lock together. Its mixed phase checks that writers never overlap readers or other writers and that two writers complete exactly 6,000 updates. The tests use atomic counters for observations; the protected payload remains ordinary memory so ThreadSanitizer can detect a synchronization failure.

Reproduce measurements with python3 scripts/run_experiments.py --environment native --cores 10 --runs 3, or run the same script with --environment docker inside the course container. Compile and run checks with python3 scripts/verify.py; for the sanitizer use --sanitizer --compiler clang++. Raw counts, histograms, compiler versions, and test logs are in results/.

Part 3: A Concurrent Linked List

Design

All three stacks keep the original representation: start points to the first node, and the last node is the stack top. Every nonempty operation begins at start and traverses next pointers to the tail; there is no cached tail pointer and no reversal of the list. A node stores its integer value and next pointer. push appends, pop removes the last node, and peek reads the last node. The allowed input values are 0, 1, and 2.

The coarse stack holds one std::mutex for each complete push, pop, or peek, including traversal. The reader-writer stack uses one std::shared_mutex: push and pop hold it exclusively because they change links or destroy nodes, while peek holds it in shared mode so several readers can traverse together. Scoped lock objects release the lock even when an empty or single-node branch returns early. Holding the lock throughout traversal prevents a writer from deleting a node that a reader still uses.

SwapTop keeps the reader-writer stack's push, pop, and peek unchanged. It first takes a shared lock and traverses to the last node. If the list is empty, it returns; if the existing value equals the requested value, it also returns because there is no write to perform. The three-value input domain makes this shortcut useful, and the benchmark heavily favors zero. Otherwise it releases the shared lock, acquires an exclusive lock, checks for an empty list again, traverses again, and replaces the current tail's value. It never keeps a node pointer across the unlocked interval, since a concurrent pop could delete that node. The no-change case takes effect at the shared-lock observation, and the changing case takes effect at the exclusive-lock assignment.

Function	Purpose, inputs, outputs, and assumptions
Llist_node(d)	Creates a node with integer d and next = null. Nodes are allocated/linked only while holding the stack's writer/exclusive lock.
CSE113_Stack()	No input. Creates an empty stack by setting start to null, before worker threads launch.
push(p)	Requires p in {0,1,2}; returns void after appending one node at the tail.
pop()	No input; removes and returns the tail value, or -1 when empty. Handles the single-node case by clearing start.
peek()	No input; returns the tail value without removing it, or -1 when empty.
swaptop(v)	Only in the third stack. Requires v in {0,1,2}; returns void after replacing the current top, or does nothing when empty.

Experiment workflow and correctness

Each stack ran exactly three times per environment, starting with 1,024 nodes. Coarse and RW use four peekers, one popper, and one pusher. SwapTop adds four swaptoppers, so it has ten workers rather than six. Counts include attempted empty operations, not just successful pops. The benchmark generates zero with probability 997/1000 and one otherwise; value two is covered by correctness tests.

The tests check empty behavior, all legal values, LIFO order, changing and unchanged SwapTop calls, preservation of 600 concurrently pushed nodes, and mixed concurrent operations. The supplied sequential checks and Clang ThreadSanitizer wrappers also pass in the course container.

Part 3: Results and Analysis

Results

Mean operation counts over three runs; parentheses give each operation's minimum and maximum. Totals include all operation types for that stack. A dash means SwapTop is not part of the implementation.

Course Docker

Stack	Total	Peek	Pop	Push	SwapTop
Coarse	255,051 (172,401 - 419,315)	171,223 (115,798 - 280,690)	41,714 (27,753 - 69,586)	42,114 (28,501 - 69,039)	-
Rw	458,722 (447,236 - 470,431)	373,704 (359,904 - 385,104)	42,407 (34,970 - 56,125)	42,612 (36,162 - 54,402)	-
Swaptop	566,383 (498,362 - 602,082)	361,492 (350,533 - 367,143)	16,532 (10,492 - 19,702)	17,151 (11,902 - 19,997)	171,208 (125,435 - 196,325)

Native macOS

Stack	Total	Peek	Pop	Push	SwapTop
Coarse	513,258 (503,952 - 519,932)	332,911 (328,979 - 335,472)	90,172 (84,285 - 93,439)	90,175 (84,195 - 93,473)	-
Rw	305,914 (288,467 - 328,957)	111,389 (102,746 - 122,631)	97,330 (92,988 - 103,301)	97,195 (92,733 - 103,025)	-
Swaptop	279,852 (239,796 - 312,148)	88,364 (72,098 - 102,113)	51,313 (48,017 - 53,301)	51,317 (48,218 - 53,303)	88,858 (71,463 - 103,431)

Analysis

In Docker, the RW stack's mean total is 1.80 times the coarse stack's, and its mean peek count is 2.18 times as large. This agrees with the expected benefit of allowing four peekers to traverse at once. On the native host, the RW/coarse total ratio is 0.60, showing that the exact benefit depends on the implementation and scheduler. Each traversal is still O(list length), so changes in stack depth can substantially change the cost of later operations. Docker coarse trial 1 reports 419,315 operations, compared with 173,436 and 172,401 in trials 2 and 3. This is a large spread, so the three-run average should not be treated as a precise stable rate; all three values remain in the average.

The SwapTop shortcut can share the read lock with peekers when the requested value already matches the tail. The benchmark's strong preference for zero makes this case more common than a uniform choice among three values would, so its throughput is specific to this workload. Actual changes require exclusive access and a second traversal; that adds work and competition with push and pop. The reported per-operation ranges show substantial variation in some runs. SwapTop also adds four workers and a new counted operation, so comparing its total directly with the six-worker stacks would confound algorithm behavior with workload size. Atomic benchmark counters, random-value generation, sleeping, background scheduling, and changing stack depth all contribute to the observed rates.